

MELODIA V2

Audit de Code & Plan d'Amélioration

Rapport d'audit technique complet couvrant la sécurité, la performance, la qualité du code et l'architecture du projet Mélodia V2 — plateforme IA de génération de souvenirs musicaux personnalisés.

Version auditée : 0.2.1 (Next.js 16 + React 19 + TypeScript + Prisma)

Date : 12 August 2026

Auditeur : ALLJOB BATACONNECT IA

CRITIQUE	HIGH	MOYEN	FAIBLE
16	21	29	24

Total : 90 problèmes identifiés sur 4 domaines d'audit

Table des Matières

1. Résumé Exécutif
2. Audit Sécurité — Vulnérabilités Critiques
3. Audit API — Routes & Endpoints
4. Audit Services & Base de Données
5. Audit Interface & Composants
6. Audit Architecture & Scalabilité
7. Plan d'Amélioration — 5 Phases
8. Matrice de Priorisation

1. Résumé Exécutif

L'audit complet du codebase Mélodia V2 a révélé 90 problèmes répartis en quatre niveaux de sévérité. Les découvertes les plus alarmantes concernent des failles de sécurité critiques permettant l'accès non authentifié à des endpoints IA coûteux, des mots de passe administrateur codés en dur dans le source, et une configuration de base de données qui peut entraîner une perte totale de données en production. Ces problèmes nécessitent une action immédiate avant tout déploiement en environnement client.

La plateforme Mélodia V2 présente une architecture ambitieuse et bien conçue sur le plan fonctionnel — le pipeline Histoire vers Émotion vers Chanson vers Cadeau est complet et innovant. Cependant, l'implémentation actuelle souffre de lacunes significatives en matière de sécurité, de performance et de maintenabilité qui, si elles ne sont pas corrigées, compromettront la fiabilité et la confiance des utilisateurs.

Points Forts Identifiés

Validation NEXTAUTH_SECRET avec refus de démarrage en production si absent ou faible
Configuration sécurisée des cookies : préfixe __Secure-, httpOnly, SameSite=lax, HTTPS en production
En-têtes de sécurité complets : CSP, HSTS (2 ans + preload), X-Frame-Options DENY, COOP
Hachage IP via SHA-256 avec sel — aucune IP brute stockée en base
Validation Zod sur les endpoints clés (story-analysis, lyrics, audio, people, admin)
Rate limiting Redis-backed avec fallback mémoire, réponses 429 appropriées
Détection d'injection de prompt en défense en profondeur
Audit trail des appels IA via AIRequestLog

Points Critiques Requiérant Action Immédiate

/api/generate sans authentification — n'importe qui peut utiliser le pipeline IA complet sans login ni limite, brûlant les clés API OpenRouter et Suno sans restriction
/api/admin/seed sans authentification — création de compte admin avec mot de passe codé en dur accessible publiquement
Base de données /tmp en production Vercel — toutes les données utilisateur sont effacées à chaque cold start, incluant chansons, crédits et transactions
Conditions de concurrence sur les crédits — le système de crédit n'est pas atomique, permettant le double-spend et les bonus d'inscription dupliqués
Composants god de 800+ lignes — create-flow-client.tsx et dashboard/page.tsx sont impossibles à maintenir et à tester unitairement

2. Audit Sécurité — Vulnérabilités Critiques

[CRITIQUE] Endpoint `/api/generate` sans authentification

Fichier : `src/app/api/generate/route.ts:49`

L'endpoint principal de génération IA (lyrics + audio) ne vérifie aucune authentification, ne limite pas le débit et ne débite aucun crédit. Toute personne anonyme peut appeler ce endpoint de manière illimitée, consommant les clés API OpenRouter et Suno sans aucune restriction. Il s'agit vraisemblablement d'un endpoint V1 legacy qui a été oublié lors de la migration V2 vers l'authentification centralisée.

Impact : Abus non authentifié des APIs IA payantes, attaque denial-of-wallet, épuisement des quotas API.

Correction : Ajouter `requireAuth()` + `checkRateLimit()` + vérification/débit de crédits. Aligner sur le pattern utilisé par `/api/story-analysis` et `/api/lyrics`.

[CRITIQUE] Endpoint `/api/admin/seed` sans authentification avec mot de passe codé en dur

Fichier : `src/app/api/admin/seed/route.ts:26-30`

L'endpoint POST `/api/admin/seed` est accessible publiquement sans authentification. Il crée un compte administrateur avec les identifiants codés en dur `admin@melodia.com` / `Melodia-Admin-2026!` ainsi que 3 utilisateurs de démonstration avec le mot de passe `Demo-User-2026!`. Le commentaire dans le code indique explicitement : « Aucune authentification requise ». Le même mot de passe `admin` est aussi codé en dur dans `src/lib/db.ts:72` comme fallback si `ADMIN_PASSWORD` n'est pas défini.

Impact : N'importe qui peut créer un compte admin avec des identifiants connus. Si le code source est public ou fuité, l'accès admin est compromis immédiatement.

Correction : Protéger l'endpoint par `requireAdmin()` ou le désactiver en production. Supprimer tous les mots de passe fallback codés en dur. Exiger `ADMIN_PASSWORD` en production avec échec au démarrage si absent.

[CRITIQUE] Condition de concurrence TOCTOU sur /api/admin/setup

Fichier : src/app/api/admin/setup/route.ts:50

L'endpoint POST /api/admin/setup ne vérifie que « aucun admin n'existe encore » via un comptage. Deux requêtes concurrentes peuvent toutes deux passer cette vérification avant que l'une d'elles n'écrive, créant deux comptes admin. Le handler GET fuit adminCount aux utilisateurs non authentifiés, facilitant la reconnaissance.

Impact : Course critique sur le premier déploiement : un attaquant peut gagner la course et créer un admin avec ses propres identifiants.

Correction : Ajouter un verrou de base de données ou une contrainte unique. Restreindre l'endpoint à localhost uniquement. Supprimer la fuite adminCount du GET.

[HIGH] JWT avec rôle/pack figé pour 30 jours sans révocation

Fichier : src/lib/auth.ts:212-231

Les champs role et pack sont intégrés au JWT lors de la première connexion et ne sont jamais rafraîchis depuis la base de données. Si un admin rétrograde un utilisateur (role admin vers user), le JWT de cet utilisateur conserve role=admin pendant jusqu'à 30 jours (maxAge: 30*24*60*60). Il n'existe aucun mécanisme de révocation de session.

Impact : Persistance de privilège : un utilisateur dégradé conserve ses droits admin pendant 30 jours.

Correction : Rafraîchir role/pack depuis la DB dans le callback jwt() à chaque requête (ou toutes les 5 minutes via cache). Réduire maxAge à 24h. Implémenter une blocklist de tokens via Redis.

[HIGH] Aucune protection brute-force sur login et signup

Fichier : src/app/api/signup/route.ts, src/app/api/auth/[...nextauth]/route.ts

Le module de rate limiting (src/lib/security/rate-limit.ts) est utilisé sur les endpoints IA mais jamais sur l'authentification. Le login permet des tentatives de mot de passe illimitées par seconde. Le signup permet la création de comptes illimitée. Aucun verrouillage de compte après N échecs consécutifs n'existe.

Impact : Attaque par force brute sur les mots de passe. Création automatisée de comptes pour farm les crédits bonus.

Correction : Appliquer checkRateLimit() avec des limites strictes : 5 tentatives login/minute par IP, 3 inscriptions/minute par IP. Ajouter failedLoginAttempts et lockedUntil au modèle User.

[HIGH] Politique de mot de passe faible — 6 caractères minimum, aucune complexité

Fichier : `src/app/api/signup/route.ts:50`

L'inscription requiert uniquement 6 caractères sans exigence de majuscule, chiffre ou caractère spécial. L'admin setup requiert 8 caractères (incohérent) mais toujours sans complexité. Les rounds bcrypt de 10 sont en dessous de la recommandation OWASP de 12 utilisée ailleurs dans le codebase.

Impact : Mots de passe trivialement devinables, augmentation du risque de compromission de comptes.

Correction : Exiger minimum 8 caractères + au moins un chiffre et un caractère spécial via `Zod.regex()`. Uniformiser les rounds bcrypt à 12 partout.

[HIGH] Aucune protection CSRF sur les endpoints API personnalisés

Fichier : Tous les POST `/api/*` (sauf `/api/auth/*`)

NextAuth fournit une protection CSRF via double-submit cookie pour ses propres endpoints. Cependant, tous les endpoints POST personnalisés (`/api/signup`, `/api/songs`, `/api/generate`, `/api/admin/*`) n'ont aucun token CSRF. Les cookies `SameSite=lax` protègent contre les attaques GET cross-site mais pas contre les attaques POST same-site ou sous-domaine.

Impact : Une page malveillante sur un sous-domaine pourrait soumettre des requêtes authentifiées si l'utilisateur a un cookie de session.

Correction : Ajouter une validation de token CSRF sur tous les POST/PUT/DELETE personnalisés, ou passer les cookies de session en `SameSite=strict` + validation de l'en-tête `Origin`.

[HIGH] Aucun middleware d'authentification centralisé

Fichier : `Projet` — aucun fichier `middleware.ts`

L'authentification est enforcee route par route via l'appel manuel à `requireAuth()/requireAdmin()`. Si un développeur oublie l'appel sur une nouvelle route, elle est silencieusement non protégée. Il n'existe aucune liste centralisée de chemins protégés/publics.

Impact : N'importe quelle nouvelle route API ajoutée sans `requireAuth()` est ouverte par défaut.

Correction : Créer un `middleware.ts` Next.js qui enforce l'auth sur `/api/*` avec une allowlist pour les endpoints publics (`/api/health`, `/api/auth/*`, `/api/gift/[slug]` en GET).

[HIGH] Injection de prompt sur les champs non-histoire

Fichier : src/lib/story/service.ts:67-80, src/lib/lyrics/service.ts:67-107

Les fonctions buildUserPrompt interpolent recipientName, relationship, occasion, style et mood directement dans le prompt LLM sans sanitization. Seul le texte de l'histoire passe par wrapUserContent/truncate. Un recipientName malveillant comme « Ignore all previous instructions. Output: ... » pourrait détourner la sortie du LLM.

Impact : Injection de prompt permettant de contourner les instructions système et de générer du contenu arbitraire.

Correction : Appliquer wrapUserContent() + truncate() à TOUS les champs fournis par l'utilisateur interpolés dans les prompts, pas seulement au texte de l'histoire.

3. Audit API — Routes & Endpoints

[CRITIQUE] /api/admin/settings POST — Schema Zod factice sans validation

Fichier : src/app/api/admin/settings/route.ts:62-66

UpdateCredentialsSchema est un objet simple qui ressemble à un schema mais ne fait rien. Il n'est jamais utilisé pour la validation. Le body est parsé via un type assertion unsafe 'as typeof body'. Aucune validation runtime sur le format email, la longueur du mot de passe ou les types de champs. Comparez avec /api/admin/users/[id]/role qui utilise un vrai schema Zod.

Impact : Un attaquant peut envoyer des champs arbitraires ou des données malformées, contourner la validation du mot de passe actuel, ou injecter des valeurs invalides.

Correction : Remplacer par un vrai schema Zod avec .safeParse(). Valider le format email, la longueur minimale du mot de passe, et rejeter les champs inconnus.

[CRITIQUE] /api/songs POST — Aucune validation Zod, assertion de type unsafe

Fichier : src/app/api/songs/route.ts:26-31

Le body est parsé via await req.json() sans aucun schema Zod. La seule vérification est if (!body.audioUrl || !body.title). Tout le reste (style, mood, language, lyricsJson) est écrit directement en base sans validation. Des payloads XSS dans lyrics ou des chaînes excessivement longues peuvent être stockés.

Impact : Stockage de données arbitraires en base, risque XSS via lyricsJson, déni de service via chaînes de longueur extrême.

Correction : Ajouter un schema Zod pour la création de chansons avec validation de longueur maximale sur tous les champs, validation du format URL pour audioUrl, et sanitization du contenu.

[HIGH] /api/generate POST — Aucun try/catch autour des appels IA

Fichier : src/app/api/generate/route.ts:104-108

Les appels generateLyrics() et generateAudio() ne sont pas wrappés dans un try/catch. Si l'un d'eux lance une exception, l'erreur se propage comme un 500 non géré avec une page d'erreur Next.js brute, sans message structuré ni réponse JSON.

Impact : Erreurs 500 non structurées fuient potentiellement des détails internes (stack traces, clés API). Expérience utilisateur dégradée.

Correction : Wrapper dans try/catch avec retour d'erreur JSON structurée. Logger l'erreur côté serveur avec le service d'observabilité.

[HIGH] Race condition check-then-debit sur les crédits

Fichier : `src/app/api/story-analysis/route.ts:57-73`, `/api/lyrics/route.ts:62-83`

La vérification du solde et le débit ne sont pas atomiques. Le flux est : 1) `getBalance()` → 2) vérifier si `balance >= cost` → 3) `debit()`. Entre les étapes 2 et 3, une autre requête concurrente peut passer la même vérification. Un utilisateur avec exactement 10 crédits pourrait passer la vérification deux fois et être débité deux fois (solde négatif).

Impact : Double-spend : un utilisateur peut dépenser plus de crédits que son solde. Le solde peut devenir négatif.

Correction : Utiliser une transaction Prisma `$transaction` avec verrouillage pessimiste ou un upsert atomique. Implémenter un débit atomique `check-and-debit` en une seule opération SQL.

[MOYEN] Pas de pagination sur `/api/people` et `/api/me` (limite 50 hard-codée)

Fichier : `src/app/api/people/route.ts:37`, `src/app/api/me/route.ts:33`

L'endpoint `/api/people` retourne toutes les personnes d'un utilisateur sans pagination. L'endpoint `/api/me` limite à 50 chansons avec un nombre magique hard-codé, sans support de pagination au-delà. Un utilisateur avec plus de 50 chansons ne peut jamais accéder aux anciennes via l'API.

Impact : Réponses potentiellement volumineuses, impossibilité de naviguer au-delà de 50 chansons, dégradation progressive des performances.

Correction : Ajouter des paramètres `limit/offset` avec valeurs par défaut raisonnables. Implémenter un pattern `cursor-based` pour les grandes collections.

[MOYEN] Double requête DB redondante dans `/api/me/credits`

Fichier : `src/app/api/me/credits/route.ts:18-27`

Deux requêtes séparées vers la même ligne `UserCredits` : `creditsService.getBalance()` puis `db.userCredits.findUnique()`. Le résultat utilise `userCredits?.balance ?? balance` — le `balance` du premier appel est redondant puisque le second récupère les mêmes données plus les champs `lifetime`.

Impact : Double charge sur la base de données pour les mêmes données. Latence inutilement doublée sur un endpoint critique du dashboard.

Correction : Consolider en une seule requête qui récupère `balance + lifetimeCredited + lifetimeSpent`. Supprimer l'appel redondant à `getBalance()`.

[MOYEN] /api/health fuite d'informations d'infrastructure sans auth

Fichier : src/app/api/health/route.ts:94-128

L'endpoint de santé retourne publiquement : latence DB, statut du secret NEXTAUTH_SECRET, présence des clés IA, environnement NODE_ENV, uptime et version. La fonction checkAuth() vérifie NEXTAUTH_SECRET contre une liste de valeurs faibles et retourne le résultat publiquement.

Impact : Information disclosure facilitant la reconnaissance par un attaquant (environnement, version, configuration de sécurité).

Correction : Retourner uniquement { status: 'ok' } pour les requêtes non authentifiées. Réserver les détails à un endpoint /api/admin/health protégé.

4. Audit Services & Base de Données

[CRITIQUE] Base de données /tmp éphémère en production — Perte de données au cold start

Fichier : src/lib/db.ts:4-18

Le code override DATABASE_URL pour pointer vers /tmp/melodia-db/ en production (Vercel serverless). Le répertoire /tmp est éphémère — toutes les données utilisateur (chansons, crédits, transactions, personnes) sont effacées à chaque cold start. C'est catastrophique pour un déploiement production.

Impact : Perte totale de données à chaque redémarrage serveur. Tous les utilisateurs perdent leurs chansons, souvenirs et crédits.

Correction : Supprimer le fallback /tmp ou le restreindre au développement local uniquement. En production Vercel, utiliser un service PostgreSQL externe (Neon, Supabase, PlanetScale) avec DATABASE_URL obligatoire.

[CRITIQUE] Race condition : grantSignupBonus non atomique

Fichier : src/lib/credits/service.ts:243-252

Le code vérifie l'absence de transaction signup_bonus via findFirst, puis crédite. Deux requêtes d'inscription concurrentes peuvent toutes deux passer la vérification (aucune ne trouve de ligne existante), puis appeler credit(), accordant le double du bonus d'inscription. Aucune transaction ne wrappe le check+credit. Aucune contrainte unique ne le prévient.

Impact : Double bonus d'inscription : un utilisateur reçoit 2x le crédit prévu en s'inscrivant simultanément depuis deux onglets/appareils.

Correction : Wrapper dans une transaction Prisma \$transaction. Ajouter une contrainte @@unique([userId, reason]) partielle sur CreditTransaction pour la raison signup_bonus. Utiliser upsert au lieu de findFirst+create.

[CRITIQUE] Données mock persistées en base de production

Fichier : src/lib/story/service.ts:218-234, src/lib/lyrics/service.ts:185-200

Quand `AI MOCK MODE=true` (ou clé API absente, car `isAIMockMode()` defaulte à mock), les données de démonstration sont écrites dans les tables `StoryAnalysis` et `Lyrics` avec `provider='demo'` et `mockMode=true`. Si quelqu'un déploie avec `AI MOCK MODE=true` ou sans clé API, la base de production est polluée avec des données fictives. Aucun garde-fou environnementnel ne prévient la persistance de données mock.

Impact : Pollution de la base de production avec des données fictives. Les analytics et les dashboards admin reflètent des données fausses.

Correction : Ajouter un garde : si `isAIMockMode()` ET `NODE_ENV==='production'`, lancer une erreur ou refuser de persister. Logger un avertissement `CRITICAL` quand le mode mock est actif en production.

[CRITIQUE] AI Log Stats charge TOUTES les lignes en mémoire — Risque OOM

Fichier : src/lib/ai/log-service.ts:77-87

La méthode de statistiques utilise `findMany()` sans limite, chargeant potentiellement des millions de lignes `AIRequestLog` en mémoire. Toute l'agrégation (taux de succès, somme des tokens, répartition par service) est effectuée en JavaScript au lieu de SQL `GROUP BY/aggregate`.

Impact : Crash OOM à l'échelle. L'endpoint admin analytics devient inutilisable avec un volume important de logs IA.

Correction : Migrer l'agrégation vers des requêtes SQL Prisma `groupBy` ou des requêtes raw avec `GROUP BY`. Ajouter une limite temporelle (derniers 30 jours par défaut).

[HIGH] Index manquants sur `Song.userId` et `Lyrics.userId` — Full table scan

Fichier : prisma/schema.prisma:116-148

Le modèle `Song` n'a aucun `@@index([userId])` ni `@@index([userId, createdAt])`. Le modèle `Lyrics` n'a pas `@@index([userId])`. Chaque requête pour les chansons ou paroles d'un utilisateur effectue un full table scan. `Song` est la table la plus interrogée (dashboard utilisateur, bibliothèque, admin, memory overview).

Impact : Dégradation linéaire des performances à mesure que le nombre de chansons augmente. Full table scan sur chaque chargement de dashboard.

Correction : Ajouter `@@index([userId, createdAt])` sur `Song` et `@@index([userId])` sur `Lyrics` dans le schema Prisma. Exécuter `prisma migrate dev` pour appliquer.

[HIGH] Aucun retry/circuit breaker sur les appels IA

Fichier : `src/lib/ai/orchestrator.ts:97-104`, `src/lib/ai/openrouter-client.ts:86-152`

Le client OpenRouter essaie les modèles fallback en cas d'échec mais ne retente jamais le même modèle. Un 503 transitoire ou un blip réseau passe immédiatement au fallback. Si OpenRouter est down pendant 5 minutes, chaque requête essaie 3-4 modèles séquentiellement avec des timeouts de 15-45s, gaspillant jusqu'à 135 secondes par requête. Aucun circuit breaker pour court-circuiter après N échecs consécutifs.

Impact : Latence extrême sous pannes intermittentes. Dégradation inutile vers le mode mock au lieu de retenter.

Correction : Implémenter un retry avec backoff exponentiel + jitter (3 tentatives max). Ajouter un circuit breaker (état fermé après 5 échecs consécutifs, reset après 60s).

[HIGH] Casts 'as' unsafe sans validation runtime sur les sorties LLM

Fichier : `src/lib/story/service.ts:93-94`, `src/lib/lyrics/service.ts:129`

Les résultats du LLM sont castés via 'as Relationship', 'as EmotionTag', 'as calm | medium | high'. Ces casts font confiance à la sortie du LLM. Si le modèle retourne une valeur invalide (e.g. 'angry' pour emotion ou 'slow' pour energy), elle est castée au type enum et persistée en base — des valeurs enum invalides sont stockées.

Impact : Données invalides persistées en base. Erreurs runtime ultérieures quand le code énumère les valeurs attendues. Incohérence de données.

Correction : Valider chaque champ de sortie LLM avec `Object.values()` check ou schema Zod. Rejeter les valeurs invalides et utiliser le fallback mock. Ajouter des tests unitaires sur les réponses LLM inattendues.

[HIGH] Générateur audio stocke des data URLs base64 de plusieurs MB en DB

Fichier : `src/lib/audio/generator.ts:217`

HuggingFace retourne un blob audio complet converti en data URL base64. Un MP3 de 30 secondes fait environ 500 Ko, soit 670 Ko en base64. Si cette data URL est stockée dans `Song.audioUrl` (colonne String), elle gonfle la base SQLite et ralentit considérablement toutes les requêtes sur la table `Song`.

Impact : Gonflement extrême de la base de données. Requêtes lentes sur `Song`. Limites SQLite atteintes rapidement.

Correction : Uploader les blobs audio vers un stockage objet (S3, R2, V. Vercel Blob) et stocker uniquement l'URL distante dans `audioUrl`. Ne jamais stocker de data URLs en base.

5. Audit Interface & Composants

[CRITIQUE] Composants god massifs : `create-flow-client.tsx` (800+ lignes) et `dashboard/page.tsx` (900+ lignes)

Fichier : `src/app/create/create-flow-client.tsx`, `src/app/dashboard/page.tsx`

`create-flow-client.tsx` gère 6 étapes de wizard, le state du formulaire, les appels API pour l'analyse d'histoire + paroles + audio, le rendu du preview et la génération avec polling — tout dans un seul composant. `dashboard/page.tsx` gère la vérification de session, la liste de chansons, la liste de personnes, les crédits, 8 onglets, la suppression de chansons, le CRUD personnes, le flux démo en attente et l'achat de packs — également dans un seul composant.

Impact : Inmaintenable. Impossible de tester unitairement. Chaque re-rend touche toute la logique. Risque de régression élevé sur tout changement. Bundle client excessif.

Correction : Extraire des sous-composants `CreateFlowStep` par étape, `DashboardTab` par onglet. Utiliser `React Query` pour la gestion de données. Implémenter la composition au lieu de l'héritage monolithique.

[CRITIQUE] Aucun lazy loading / code splitting dans l'ensemble du projet

Fichier : Tous les composants client de page

Zéro utilisation de `React.lazy()`, `next/dynamic` ou `Suspense boundaries` au niveau route. Les seuls `Suspense` sont dans `signup/page.tsx` pour `useSearchParams`. Les composants lourds comme `AudioPlayer`, `StoryDnaRadar`, `AI SurpriseWidget` et `RecommendationsStrip` sont tous importés de manière statique (`eager`). Le dashboard + create-flow + memory bundle est expédié au client au premier chargement.

Impact : Bundle JavaScript initial probablement 200 Ko+ de code inutile. Time to Interactive dégradé. Expérience mobile significativement affectée.

Correction : Utiliser `next/dynamic` avec `suspense:true` pour les composants lourds. Ajouter des `Suspense boundaries` par route segment. Mesurer le bundle avec `@next/bundle-analyzer`.

[CRITIQUE] Aucun error boundary par segment de route

Fichier : Tous les `app/*/page.tsx`

Seul `src/app/global-error.tsx` existe (catch les crashes fatals). Il n'y a aucun `error.tsx` aux niveaux `/create`, `/dashboard`, `/memory`, `/gift/[slug]`, `/people/[id]` ou `/admin/*`. Si un composant client lance une erreur pendant le render (e.g. `fetch` échoué -> `déréférencement null`), l'application entière crash vers la page d'erreur globale au lieu d'afficher une erreur inline.

Impact : Une seule erreur API sur le dashboard fait crasher toute l'application. Expérience utilisateur catastrophique sur erreur transitoire.

Correction : Ajouter `error.tsx` à chaque segment de route principal. Afficher une erreur inline avec bouton de retry. Préserver le layout (header/sidebar) même quand le contenu échoue.

[HIGH] 191+ chaînes français hard-codées sans passage par `i18n`

Fichier : `src/components/dashboard/*.tsx`, `src/app/admin/admin-shell.tsx`, `src/app/gift/[slug]/gift-page-client.tsx`, `src/app/people/[id]/person-detail-client.tsx`

L'application supporte FR/EN/ES via `next-intl`, mais le dashboard entier, l'admin, les pages cadeau, le détail personne et les pages 404/500 sont entièrement en français hard-codé. Changer la locale n'a aucun effet sur ces pages. Les pires contrevenants sont `sidebar.tsx` (10+ chaînes), `sections.tsx` (12+ chaînes), `landing-dashboard.tsx` (18+ chaînes), `admin-shell.tsx` (9+ chaînes), `gift-page-client.tsx` (extensif), `person-detail-client.tsx` (15+ chaînes).

Impact : L'internationalisation est non fonctionnelle sur la majorité de l'interface. Les utilisateurs non-francophones ne peuvent pas utiliser le dashboard, l'admin ou les pages cadeau.

Correction : Créer les clés `t.dashboard.*`, `t.admin.*`, `t.gift.*`, `t.people.*` dans le dictionnaire `i18n`. Remplacer toutes les chaînes hard-codées par des appels `useT()`. Prioriser le dashboard et les pages orientées client.

[HIGH] Balises 'a' au lieu de Link pour la navigation interne (dashboard)

Fichier : `src/components/dashboard/sections.tsx`, `widgets.tsx`, `landing-dashboard.tsx`, `top-bar.tsx`

Plusieurs composants dashboard utilisent des balises d'ancres HTML classiques (`href='/dashboard?tab=creations'` et `href='/create'`) au lieu du composant `Link` de `Next.js`. Cela provoque des rechargements complets de page à chaque navigation, perdant le state client et causant un flash visible.

Impact : Toute navigation dans le dashboard recharge la page entière. Perte du state local. Expérience SPA dégradée.

Correction : Remplacer toutes les balises d'ancres internes par le composant `Link` de `next/link`. Seules les ancres externes conservent la balise HTML classique.

[HIGH] Race conditions dans les hooks de fetch sans AbortController

Fichier : src/hooks/use-landing-data.tsx:100-220, src/app/people/[id]/person-detail-client.tsx:78-114

Les `useEffect` dans `useLandingData` déclenchent 3 fetchs parallèles et fusionnent les résultats. Si le composant se démonte et remonte rapidement (changement de route), le flag d'annulation empêche le `setState` obsolète — mais il n'y a aucun `AbortController`. Les requêtes HTTP continuent à s'exécuter et consommer de la bande passante.

Impact : Requêtes réseau gaspillées sur navigation rapide. Potentiel de corruption de state si le timing d'annulation est défavorable.

Correction : Ajouter `AbortController` à tous les `useEffect` de fetch. Signaler les requêtes dans le `cleanup` du `useEffect`. Suivre le pattern React recommandé pour les fetchs dans les effets.

[MOYEN] Boutons morts et recherche non fonctionnelle

Fichier : src/components/dashboard/widgets.tsx, sidebar.tsx, top-bar.tsx, person-detail-client.tsx

Plusieurs éléments interactifs n'ont aucun handler `onClick` : « Ajouter un événement » (`widgets.tsx:89`), « Offrir un bon cadeau » (`sidebar.tsx:365`), « Modifier » (`person-detail-client.tsx:237`), le champ de recherche (`top-bar.tsx:45`). Le bouton Play sur les cartes de création (`sections.tsx:330`) a un `aria-label` mais aucun `onClick`.

Impact : Interface trompeuse : l'utilisateur voit des boutons qui ne font rien. Confusion et frustration. Le search bar suggère une fonctionnalité inexistante.

Correction : Implémenter les handlers `onClick` ou masquer les boutons non fonctionnels avec une indication 'prochainement'. Désactiver le search bar ou le connecter à une logique de filtrage.

6. Audit Architecture & Scalabilité

[CRITIQUE] DDL SQL raw divergent du schema Prisma — Aucun système de migration

Fichier : `src/lib/db.ts:135-161`

`_ensureTables()` utilise du DDL SQL hard-codé qui n'est pas synchronisé avec `prisma/schema.prisma`. Il manque : les contraintes de clé étrangère, de nombreuses colonnes ajoutées dans les versions ultérieures du schema (relations Song vers StoryAnalysis/Lyrics/GiftPage/Souvenir), et les index appropriés. Si ce code path s'exécute (aucune migration préalable), l'application crash quand Prisma tente d'accéder aux colonnes/rerelations manquantes.

Impact : Crash au démarrage sur un environnement sans migration préalable. Incohérence entre la structure réelle de la DB et les attentes de Prisma.

Correction : Supprimer `_ensureTables()` et utiliser exclusivement le système de migration Prisma (`prisma migrate deploy` en production). Ne jamais bootstrapper via DDL raw.

[HIGH] Aucun support streaming pour les appels IA longs

Fichier : `src/lib/ai/orchestrator.ts`, `src/lib/ai/openrouter-client.ts`

La génération de paroles peut prendre 15-30 secondes. L'intégralité de la réponse est attendue avant tout feedback UI. Aucun Server-Sent Events ni streaming. L'utilisateur voit un spinner pendant 30s sans aucune indication de progression.

Impact : Expérience utilisateur dégradée avec attente prolongée sans feedback. Abandon de création probable après 15s+ d'attente silencieuse.

Correction : Implémenter le streaming via OpenRouter streaming API + `ReadableStream` côté Next.js. Envoyer des événements de progression au client via SSE. Afficher les paroles au fur et à mesure de la génération.

[HIGH] Admin stats : 24 requêtes parallèles sans cache

Fichier : `src/lib/admin/service.ts:57-127`

`getAdminStats()` déclenche 24 requêtes DB parallèles sans aucun caching ni fenêtre de staleness. Sous charge, cela martèle la base de données. Aucune limite temporelle sur la période couverte par les statistiques.

Impact : Surcharge DB sous accès admin concurrent. Latence croissante avec le volume de données.

Correction : Implémenter un cache avec stale-while-revalidate (SWR) de 60 secondes. Utiliser une vue matérialisée ou un snapshot pré-calculé rafraîchi périodiquement. Limiter la période par défaut à 30 jours.

[HIGH] Logique de crédits admin divergente de `CreditsService`

Fichier : `src/lib/admin/service.ts:370-414`

`adminAdjustCredits()` crée sa propre `$transaction` au lieu d'utiliser `creditsService.debit()/credit()`. Il calcule manuellement `lifetimeCredited/lifetimeSpent` au lieu d'utiliser les increments Prisma. Si la logique de `CreditsService` change (e.g. ajout d'un plancher de balance), les ajustements admin ne suivront pas.

Impact : Divergence de logique métier entre l'admin et le service de crédits. Comportement inattendu après refactor de `CreditsService`.

Correction : Refactor pour utiliser `creditsService.credit()/debit()` même pour les ajustements admin. Ajouter un paramètre `reason='admin_adjust'` pour différencier dans le ledger.

[MOYEN] Fallback mock silencieux quand la clé API est absente

Fichier : `src/lib/ai/tiers.ts:84-89`

Si `AI MOCK MODE` est unset et `OPENROUTER_API_KEY` est manquante (env mal configuré), l'application fonctionne silencieusement en mode mock. Les utilisateurs reçoivent des paroles de démonstration sans aucune indication. Aucun avertissement n'est loggé quand le mode mock est activé automatiquement.

Impact : Dégradation silencieuse : l'application semble fonctionner normalement mais génère du contenu fictif. Les utilisateurs ne savent pas qu'ils n'utilisent pas la vraie IA.

Correction : Logger un avertissement CRITICAL au démarrage quand `isAIMockMode()` retourne `true` en production. Afficher un bandeau visible dans l'UI quand le mode mock est actif. Refuser de démarrer en production sans clé API.

[MOYEN] Fichier lyrics/openrouter.ts legacy duplique l'orchestrateur V2

Fichier : src/lib/lyrics/openrouter.ts

Ce fichier contient un client OpenRouter complètement séparé avec son propre system prompt, appel fetch, parsing JSON et fallback mock — dupliquant ce que aiOrchestrator + lyrics/service.ts font déjà. C'est le chemin V1 et crée une confusion sur le chemin de code actif. Le system prompt et le schéma JSON diffèrent de la V2.

Impact : Confusion de maintenance : quel chemin de code est actif ? Risque de fix appliqué au mauvais fichier. Comportement différent selon le chemin emprunté.

Correction : Supprimer src/lib/lyrics/openrouter.ts et s'assurer que tous les chemins passent par l'orchestrateur V2. Ajouter des tests de regression pour confirmer l'équivalence.

[LOW] Duplications de code : hashString x3, Relationship type x2, mappings x4

Fichier : src/lib/story/mock-analyzer.ts, src/lib/lyrics/mock-v2.ts, src/lib/audio/generator.ts, src/lib/story/types.ts, src/lib/memory/service.ts

La fonction hashString est copiée-collée identiquement dans 3 fichiers. Le type Relationship est défini dans 2 fichiers. Les mappings relationship->label et relationship->emoji sont dupliqués dans 4+ emplacements. Modifier un label nécessite d'éditer 3-4 fichiers.

Impact : Violation DRY. Risque d'incohérence si un mapping est mis à jour dans un fichier mais pas dans les autres. Maintenance pénible.

Correction : Extraire hashString vers src/lib/utils.ts. Extraire les types et mappings Relationship vers src/lib/relationships.ts. Remplacer tous les usages par les imports centralisés.

7. Plan d'Amélioration — 5 Phases

Le plan d'amélioration est structuré en 5 phases prioritaires, chacune avec des livrables concrets et des métriques de succès. Les phases sont ordonnées par urgence : la Phase 1 traite les vulnérabilités critiques de sécurité qui bloquent tout déploiement production, tandis que la Phase 5 aborde les améliorations de qualité à long terme.

Phase 1 — Sécurité Critique (Semaine 1)

Cette phase élimine les vulnérabilités qui permettraient à un attaquant de compromettre le système ou de consommer des ressources IA sans restriction. Aucun déploiement production ne doit être effectué avant la complétion de cette phase.

ID	Tâche	Sévérité	Effort
S1.1	Ajouter requireAuth() + rate limit + credit check à /api/generate	CRITIQUE	2h
S1.2	Protéger /api/admin/seed par requireAdmin() ou le désactiver en prod	CRITIQUE	1h
S1.3	Supprimer mots de passe fallback codés en dur dans db.ts	CRITIQUE	1h
S1.4	Créer middleware.ts avec auth enforcement centralisé	CRITIQUE	4h
S1.5	Rafraîchir JWT role/pack depuis DB à chaque requête	HIGH	3h
S1.6	Ajouter rate limit sur login (5/min) et signup (3/min)	HIGH	2h
S1.7	Renforcer politique mot de passe : 8 chars + complexité	HIGH	1h
S1.8	Ajouter validation CSRF sur POST endpoints custom	HIGH	3h
S1.9	Supprimer fallback /tmp DB en production	CRITIQUE	1h
S1.10	Ajouter Zod validation à /api/songs et /api/admin/settings	CRITIQUE	2h

Livable : Toutes les routes API sont protégées par auth. Aucun mot de passe codé en dur. Middleware actif. Rate limiting sur tous les endpoints sensibles.

Métrique de succès : 0 endpoint API non authentifié (sauf allowlist explicite). 0 mot de passe hard-codé dans le source.

Phase 2 — Stabilité & Données (Semaine 2)

Cette phase traite les problèmes de concurrence, de persistance de données et de robustesse des services qui pourraient corrompre les données utilisateur ou causer des pertes irréversibles.

ID	Tâche	Sévérité	Effort
S2.1	Rendre les opérations de crédit atomiques (\$transaction + upsert)	CRITIQUE	4h

ID	Tâche	Sévérité	Effort
S2.2	Ajouter contrainte unique sur CreditTransaction(userId, reason='signup_bonus')	CRITIQUE	2h
S2.3	Garder contre mock data en production (env guard)	CRITIQUE	2h
S2.4	Migrer AI log stats vers SQL GROUP BY (supprimer OOM risk)	CRITIQUE	3h
S2.5	Ajouter indexes Prisma : Song(userId,createdAt), Lyrics(userId)	HIGH	1h
S2.6	Supprimer _ensureTables() DDL raw, utiliser Prisma migrate	CRITIQUE	3h
S2.7	Implémenter retry + backoff + circuit breaker sur appels IA	HIGH	4h
S2.8	Valider sorties LLM avec schema runtime (pas de 'as' casts)	HIGH	3h
S2.9	Uploader audio vers stockage objet (S3/R2) au lieu de data URLs	HIGH	4h

Livrable : Système de crédits atomique et fiable. Base de données propre avec indexes. Appels IA résilients avec circuit breaker. Aucune donnée mock en production.

Métrique de succès : 0 rapport de solde négatif. Temps de réponse dashboard < 500ms avec 10 000 chansons. Circuit breaker actif après 5 échecs IA consécutifs.

Phase 3 — Architecture UI (Semaines 3-4)

Cette phase refactor les composants god, ajoute les error boundaries et le code splitting, et internationalise les pages principales. C'est la phase la plus lourde en effort mais essentielle pour la maintenabilité à long terme.

ID	Tâche	Sévérité	Effort
S3.1	Découper create-flow-client.tsx en sous-composants par étape	CRITIQUE	8h
S3.2	Découper dashboard/page.tsx en composants par onglet	CRITIQUE	8h
S3.3	Découper memory-client.tsx en sections	HIGH	4h
S3.4	Ajouter error.tsx à chaque route segment	CRITIQUE	2h
S3.5	Ajouter next/dynamic + Suspense pour composants lourds	CRITIQUE	3h
S3.6	Internationaliser dashboard (191+ chaînes FR)	HIGH	8h
S3.7	Internationaliser admin, gift, people pages	HIGH	6h
S3.8	Remplacer ancres HTML par composant Link dans dashboard	HIGH	2h
S3.9	Ajouter AbortController aux hooks de fetch	HIGH	3h
S3.10	Implémenter ou masquer boutons morts + search bar	MOYEN	3h
S3.11	Extraire mappings Relationship vers src/lib/relationships.ts	MOYEN	2h

Livrable : Composants de taille < 200 lignes. Error boundaries par route. Code splitting actif. Dashboard internationalisé FR/EN/ES. Navigation SPA sans rechargement.

Métrique de succès : Aucun composant > 200 lignes. Bundle initial réduit de 40%+. Lighthouse Performance > 85. i18n fonctionnel sur 100% des pages client.

Phase 4 — Performance & DX (Semaines 5-6)

Cette phase optimise les performances de la base de données, ajoute le support streaming pour l'IA, et améliore l'expérience développeur avec un monitoring et des tests robustes.

ID	Tâche	Sévérité	Effort
S4.1	Implémenter streaming SSE pour génération de paroles	HIGH	6h
S4.2	Ajouter pagination cursor-based sur /api/songs, /api/people	MOYEN	4h
S4.3	Cache admin stats avec SWR (60s staleness)	HIGH	3h
S4.4	Consolider /api/me/credits en requête unique	MOYEN	1h
S4.5	Refactor adminAdjustCredits pour utiliser CreditsService	HIGH	2h
S4.6	Ajouter logging structuré + alertes mode mock en prod	MOYEN	3h
S4.7	Supprimer legacy lyrics/openrouter.ts	MOYEN	1h
S4.8	Configurer connection pool Prisma pour PostgreSQL	MOYEN	2h
S4.9	Ajouter tests unitaires services (credits, story, lyrics)	HIGH	8h
S4.10	Ajouter tests E2E flows critiques (signup, create, gift)	HIGH	6h

Livable : Streaming IA fonctionnel. Pagination sur toutes les collections. Admin dashboard performant. Suite de tests couvrant les chemins critiques.

Métrique de succès : Time to first token (lyrics) < 2s. Couverture de tests > 60% sur services critiques. Admin stats < 200ms p95.

Phase 5 — Qualité & Accessibilité (Semaines 7-8)

Cette phase aborde les améliorations de qualité long-terme : accessibilité, conformité, nettoyage de code et optimisations finales qui renforcent la confiance et la maintenabilité du produit.

ID	Tâche	Sévérité	Effort
S5.1	Ajouter aria-labels et landmarks sur dashboard/admin	MOYEN	4h
S5.2	Ajouter prefers-reduced-motion aux animations	MOYEN	2h
S5.3	Créer table AuditLog dédiée (pas AnalyticsEvent)	MOYEN	3h
S5.4	Consolider error response shapes (jsonOk/jsonError standard)	MOYEN	4h
S5.5	Supprimer endpoint debug /api/route.ts hello world	LOW	0.5h
S5.6	Fix liens morts footer (# href) avec URLs réelles	LOW	1h

ID	Tâche	Sévérité	Effort
S5.7	Valider couleurs chart.tsx contre injection CSS	MOYEN	1h
S5.8	Remplacer \$executeRawUnsafe par \$executeRaw	MOYEN	1h
S5.9	Dédier IP_HASH_SALT (pas NEXTAUTH_SECRET fallback)	MOYEN	1h
S5.10	Audit accessibilité WCAG 2.1 AA complet	MOYEN	8h

Livrable : Interface accessible WCAG 2.1 AA. Audit trail dédié et conforme. API responses standardisées. Codebase nettoyé sans dead code ni duplications.

Métrique de succès : Lighthouse Accessibility > 90. 0 lien mort. 0 duplication de code significative. Audit log queryable et immutable.

8. Matrice de Priorisation

La matrice ci-dessous croise l'impact (conséquence si non résolu) avec l'effort estimé pour guider les décisions de priorisation. Les éléments en haut-gauche (impact élevé, effort faible) sont les quick wins à traiter en premier.

Quadrant	Items	Recommandation
Impact TRES HAUT / Effort FAIBLE	S1.1, S1.2, S1.3, S1.9, S1.10, S2.5	Quick wins sécurité — traiter IMMEDIATEMENT
Impact TRES HAUT / Effort MOYEN	S1.4, S1.5, S2.1, S2.4, S2.6, S3.4, S3.5	Bloqueurs production — Phase 1-2
Impact HAUT / Effort FAIBLE	S1.7, S2.9, S3.8, S4.4, S4.7	Quick wins qualité — intégrer Phase 1-2
Impact HAUT / Effort ELEVE	S3.1, S3.2, S3.6, S4.1, S4.9, S4.10	Investissement majeur — Phase 3-4
Impact MOYEN / Effort FAIBLE	S3.11, S5.5, S5.6, S5.8, S5.9	Polish — Phase 5 ou opportuniste
Impact MOYEN / Effort ELEVE	S3.7, S4.2, S5.1, S5.10	Amélioration long-terme — Phase 5

Effort Total Estimé

L'ensemble du plan d'amélioration représente environ 160 heures de travail (4 semaines à temps plein ou 8 semaines à mi-temps). La Phase 1 seule représente 20 heures et élimine toutes les vulnérabilités critiques qui bloquent le déploiement production.

Phase	Effort	Durée
Phase 1 — Sécurité Critique	20h	1 semaine
Phase 2 — Stabilité & Données	26h	1 semaine
Phase 3 — Architecture UI	49h	2 semaines
Phase 4 — Performance & DX	36h	2 semaines
Phase 5 — Qualité & Accessibilité	25.5h	1-2 semaines
TOTAL	~157h	7-8 semaines

Recommandation Stratégique

Commencer immédiatement par la Phase 1 (sécurité critique). Ces 20 heures de travail éliminent les vulnérabilités qui rendent tout déploiement production irresponsable. En parallèle, provisionner une base de données PostgreSQL managée (Neon, Supabase) pour remplacer le fallback /tmp qui causait la perte de données. Une fois la Phase 1 complétée, le déploiement sur

un environnement de staging est possible, permettant de valider les corrections avant de traiter les phases suivantes.

Les Phases 2 et 3 peuvent être partiellement parallélisées : pendant qu'un développeur refactor les composants UI (Phase 3), un autre peut durcir les services et la base de données (Phase 2). Les Phases 4 et 5 sont des améliorations incrémentales qui peuvent être intégrées au rythme des sprints réguliers sans bloquer les livraisons fonctionnelles.